Midterm Project Report

Project Title: Autonomous Motion-Triggered Security Tracker

Course: CSC 126 (Graphics and Visual Computing)

Developer: Aldis S. Valdehueza, Laurence Paul Jigger R. Egay, Jiefhox G. Monter

1. Project Objectives and Description

The primary objective of this project is to develop an automated, computer vision-based security surveillance system utilizing Python and the OpenCV library. Traditional security cameras passively record hours of empty environments, requiring tedious human review and wasting vital storage space. This application solves that real-world problem by acting as an active, autonomous motion-triggered alarm system.

The application processes a live webcam feed, captures a static baseline image of an empty room, and continuously monitors for significant pixel changes. Upon detecting an intruder, the system locks onto the subject with graphical bounding boxes, displays an on-screen visual alert, and plays a continuously looping custom audio alarm (`walangkanenbuseng.wav`). To maintain autonomy and prevent the system from being indefinitely locked in an alarm state after a subject has left, it features a dynamic 3-second auto-reset protocol: if the environment remains completely still for three continuous seconds, the system automatically purges the audio stream, captures a fresh environmental baseline, and resumes quiet monitoring.

2. Core OpenCV Techniques Implemented

To achieve accurate motion tracking, minimize false positives, and ensure real-time performance, the system implements three core computer vision techniques from the CSC 126 syllabus:

A. Image Operations (Grayscale and Gaussian Blur)

Raw video frames contain massive amounts of data across three color channels (Red, Green, Blue) and often suffer from digital camera noise. To optimize processing speeds, the system immediately applies `cv2.cvtColor` to collapse the multi-channel BGR frame into a single Grayscale matrix. Subsequently, `cv2.GaussianBlur` is applied using a 21×21 kernel filter. This specific blurring algorithm acts as a low-pass filter, mathematically smoothing out erratic, high-frequency pixel noise (such as flickering lights or slight camera lens vibrations) that would otherwise trigger false motion alarms.

B. Image Analysis and Transformation (Absolute Difference and Thresholding)

Digital motion detection fundamentally relies on identifying mathematical differences between two sequential or baseline images. The system compares the current blurred frame against the pre-recorded static baseline frame using `cv2.absdiff`. The resulting difference array produces an image where unchanged background pixels approach zero (absolute black) and changed pixels (the moving subject) possess higher intensity values (gray/white).

To make the intruding subject distinctly identifiable to the tracking algorithm, `cv2.threshold` is applied. This binary transformation forces any pixel with a difference score higher than an established limit (25) to become stark white (255), while all remaining background pixels become absolute black (0). Finally, a morphological dilation (`cv2.dilate`) is passed over the threshold mask three times. This fills in any structural gaps or holes in the white silhouette, fusing fragmented pixels into a solid, cohesive block for the computer to track.

C. Motion and Object Tracking (Contours and Bounding Boxes)

Once the binary mask cleanly highlights the structural differences in stark white, the system uses `cv2.findContours` to mathematically trace the outer boundaries of these geometric shapes. The tracking algorithm filters these resulting contours by calculating their overall area using `cv2.contourArea`. In this implementation, any shape smaller than 5,000 pixels is immediately discarded as environmental noise (e.g., a bug flying near the lens or a shifting shadow). For large, valid contours,`cv2.boundingRect` calculates the spatial coordinates (x, y, width, height) of the shape and draws a highly visible, 2-pixel wide green rectangle around the subject directly onto the original full-color video feed.

3. Results and Visual Outputs

During live testing iterations, the system successfully calibrated to the background and maintained a quiet state. Upon a subject entering the frame, the OpenCV pipeline rapidly isolated the changing pixels, drew precise bounding boxes, and triggered both the on-screen alert and the asynchronous looped audio alarm.



Figure 1: The system actively isolating a moving subject from the baseline environment. The green bounding boxes represent the extracted contours, and the persistent red UI indicates the alarm loop has been triggered.

4. Discussion: Accuracy, System Behavior, and Limitations

The integration of the 3-second auto-reset timer and the custom asynchronous audio payload significantly elevates the application from a simple motion script to a robust, hands-free security tool. By utilizing the built-in Windows `winsound` library chained with the `SND_ASYNC` and `SND_LOOP` flags, the custom `walangkanenbuseng.wav` audio file plays continuously without blocking the camera's active `while` loop. This multi-threading-like behavior ensures there are zero frame drops, tracking lag, or UI freezing during an active intrusion.

The timer implementation acts as a software-based debounce mechanism. By continuously updating a `last_motion_time` variable whenever a contour larger than 5,000 pixels is detected, the system safely counts 3 seconds of absolute quiet before utilizing `SND_PURGE` to break the audio loop and recalibrate.

Limitations: Because the system relies entirely on absolute pixel differencing against a static baseline, it is highly sensitive to severe environmental lighting changes. For example, if a cloud blocks the sun and darkens the room, the entire frame's pixel values shift, causing the system to trigger a false alarm until the 3-second auto-reset safely recalibrates the baseline. Furthermore, as visible in Figure 1, strong directional lighting causes the subject's shadow to cast onto the wall behind them. Since the shadow obscures the original background pixels, the algorithm groups the shadow and the physical subject into the same spatial contour, resulting in a bounding box that is slightly wider than the actual subject.

5. Conclusion and Potential Improvements

This project successfully demonstrates the foundational principles of computer vision and image processing. By mathematically combining image operations, binary thresholding, and contour extraction, raw, chaotic pixel data can be seamlessly transformed into actionable, logical security events.

Future iterations of this software could resolve the shadow and dynamic lighting limitations by replacing the static background capture with a "Running Average" background subtraction method (using `cv2.accumulateWeighted` or OpenCV's built-in `BackgroundSubtractorMOG2`). This would allow the baseline image to dynamically and gradually adapt to lighting shifts over time. Additionally, integrating a Haar Cascade Classifier or a deep learning model could upgrade the system to track specifically human faces or bodies, rather than triggering on inanimate moving objects like a swinging door or a family pet.